A Plant Disease Local Assistant

Riccardo Terrenzi[1]

[1]University of Southern Denmark, Sønderborg, Denmark

Abstract

This report describes the design and implementation of a local plant disease assistant that combines image-based disease classification, visual explainability, retrieval-augmented plant disease advice, and an agentic user interface. The project is organized around two major preparation steps and one runtime system. First, convolutional neural network models were trained to classify leaf images into 38 plant disease or healthy classes. A custom CNN was implemented as a baseline, and a MobileNetV2 transfer-learning model was trained and fine-tuned to obtain the final default classifier. Second, a local vector database was built from curated plant pathology, crop-care, and disease-management documents, making it possible to retrieve grounded information about symptoms, prevention, and treatment. These assets were then exposed through Model Context Protocol (MCP) tools and used by a LangChain agent running on top of a local Ollama-hosted large language model. The final application provides a Streamlit chat interface where the user can upload a leaf image, receive a diagnosis, request model comparison or SHAP visual explanations, and ask follow-up questions grounded in retrieved plant disease resources.

Keywords

Agents, Large Language Models, Retrieval Augmented Generation, Convolutional Neural Network, Model Context Protocol

1. Introduction and Motivation

Plant diseases are a practical and visually observable problem: many infections, nutrient issues, and pest-related symptoms appear directly on leaves. For this reason, leaf image classification is a natural application domain for convolutional neural networks (CNNs). At the same time, a disease label alone is not enough for a useful assistant. A plant owner or grower usually needs several connected pieces of information: what the likely disease is, how confident the model is, whether there are alternative diagnoses, what visual regions influenced the prediction, what symptoms are expected, and what management or prevention actions are recommended.

The motivation for this project was therefore to build more than a standalone classifier. The goal was to create a local plant disease assistant that joins three complementary components:

1. A trained image classifier able to recognize common plant disease classes from leaf images.
2. An explainability mechanism that can show which image regions influenced a model prediction.
3. A retrieval and agent layer that can convert a prediction into useful, grounded advice using local plant disease documents.

The resulting system is designed as a local application. The image models are saved as Keras artifacts, the retrieval knowledge base is stored in a local Chroma vector database, the language model is served locally through Ollama [1], and the user interface runs in Streamlit [2].

The project deliberately separates experimental preparation from runtime execution. The notebooks document how the image models and vector database were created, while the main

Advanced Artificial Intelligence course, academic year 2025/26

rite@mmmi.sdu.dk (R. Terrenzi)

[1]https://ollama.com/

[2]https://docs.streamlit.io/

source package contains the runtime system. This separation is important because model training and document indexing are expensive or occasional tasks, whereas the final assistant should load already-prepared artifacts and answer user questions interactively.

2. Preparatory Work: From Data to Runtime Assets

Before the agentic application could be implemented, it was necessary to train CNN models for plant disease classification and create a searchable vector database containing plant disease and crop management information.

These assets correspond to two different forms of knowledge. CNN models encode visual knowledge learned from labeled leaf images. The vector database encodes textual knowledge extracted from external plant pathology and extension resources. The final assistant uses both: the classifier answers "what does this image look like?", while retrieval answers "what does this disease mean and what should be done about it?".

At runtime, these assets are treated as fixed local resources. The application does not retrain the model or rebuild the vector database every time it starts. Instead, it validates that the required assets exist, loads them lazily when needed, and exposes them to the agent through MCP tools.

3. Dataset Selection and Image Classification Setup

3.1. Image Dataset Choice

The image classifier was trained on an augmented plant disease image dataset [3]. The dataset contains labeled images for 38 classes, covering multiple crops and disease categories. The saved class list includes many fruit and vegetable plants such as apple, blueberry, or tomato classes. Some classes correspond to healthy leaves, while others correspond to specific diseases such as apple scab or cedar apple rust.

This dataset was suitable for the project for several reasons. First, it directly matches the visual diagnosis task: the input is a plant leaf image and the output is a disease or healthy class. Second, it includes a relatively broad set of crops and diseases, which makes the assistant more interesting than a single-crop classifier. Third, the class labels are semantically meaningful and can be connected to plant disease documents. For example, if the classifier predicts `Tomato___Early_blight`, the retrieval system can query local resources about tomato early blight and provide management recommendations. This connection between the classifier label space and the document knowledge base is essential for the final agent.

3.2. Image Preprocessing

Both trained models use an input size of 224×224 pixels with three color channels. During dataset loading, TensorFlow's `image_dataset_from_directory` was used with inferred labels and categorical label encoding. The relevant setup was:

The training pipeline also used TensorFlow dataset optimizations. Images and labels were mapped into `float32`, the training dataset was shuffled, and both training and validation datasets were prefetched with `tf.data.AUTOTUNE`. This is a practical performance detail: JPEG decoding and input preprocessing can otherwise become a bottleneck, especially when the model is trained on a GPU.

[3] https://www.kaggle.com/datasets/vipoooool/new-plant-diseases-dataset/data

Table 1
Dataset loading configuration.

Parameter	Value
Image size	`(224, 224)`
Label mode	`categorical`
Training directory	`train`
Validation directory	`valid`
Batch size	64 per replica, multiplied by the number of synchronized replicas
Random seed	42

Table 2
Model compilation configuration.

Setting	Value
Optimizer	Adam
Learning rate	10^{-3}
Loss	Categorical cross-entropy
Metric	Accuracy
Steps per execution	32

3.3. Data Augmentation

The same augmentation block was shared by the custom CNN and the MobileNetV2 model. It used random horizontal flipping, random rotation with factor 0.08 and random zoom with factor 0.10. The purpose of augmentation was to make the model more robust to natural variation in leaf images. A leaf may appear in different orientations, at slightly different scales, and under slightly different camera framing. Augmentation encourages the classifier to focus on disease-related visual patterns rather than memorizing exact image layouts.

4. Custom CNN Baseline

4.1. Purpose of the Baseline

The first image model was a custom CNN baseline. A baseline model is useful because it gives a reference point for evaluating whether transfer learning is actually beneficial. If a simple CNN already performs well, the additional complexity of a pretrained architecture must be justified. If the pretrained model performs substantially better, then the benefit of reusing ImageNet features becomes visible.

The custom CNN was intentionally compact. It was not designed to be the largest possible classifier, but rather to provide a controlled convolutional architecture trained directly on the plant disease dataset.

4.2. Architecture

The custom CNN accepts an input tensor of shape `(224, 224, 3)`. The architecture begins with the shared data augmentation layer, then rescales pixel values by `1/255`. It then applies a sequence of convolutional blocks, as shown in Fig.1.

The final dense layer explicitly uses `float32` output. This is relevant because the notebook optionally enables mixed precision. With mixed precision, many computations can use lower precision for speed, but the final softmax probabilities should remain numerically stable and convenient for downstream processing.

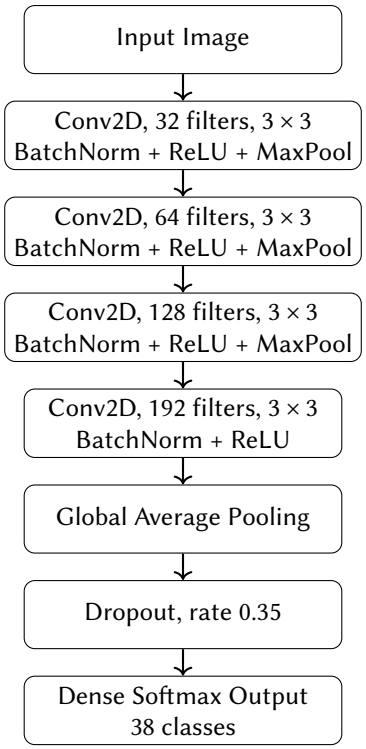

Figure 1: Convolutional neural network architecture.

4.3. Training Configuration

The custom CNN was compiled with the parameters described in Table 2.

The model was trained for up to 8 epochs with early stopping on validation accuracy, with patience 3 and best-weight restoration, and a learning-rate reduction on validation loss, with factor 0.3, patience 2, and minimum learning rate 10^{-6}.

These callbacks address two practical concerns. Early stopping prevents unnecessary training after validation performance stops improving. Learning-rate reduction allows the optimizer to take smaller steps if validation loss plateaus, which can improve convergence.

4.4. Observed Performance

The notebook output shows that the custom CNN reached approximately 93.3% validation accuracy. This is a strong result for a compact model, but it remained below the MobileNetV2 transfer-learning model. The custom CNN was still kept as a runtime model because it provides an independent comparison point. In the final application, the agent can compare predictions between the custom CNN and MobileNetV2 when uncertainty or model agreement is relevant.

5. MobileNetV2 Transfer Learning and Fine-Tuning

5.1. Why MobileNetV2

MobileNetV2 [1] was selected as the transfer-learning model because it is efficient and widely used for image classification. It is designed to provide strong visual features with relatively low computational cost. This is particularly appropriate for a local assistant: the runtime system should be able to load the model and make predictions interactively without requiring a very large architecture.

Using MobileNetV2 also introduces pretrained visual knowledge. The base model was initialized with ImageNet weights. Although ImageNet is not a plant disease dataset, its early and

intermediate layers learn generic visual features such as edges, textures, shapes, and color patterns. These features can be useful for plant images after a new classification head is trained on the target disease labels.

5.2. Frozen-Base Training

The first MobileNetV2 stage used the pretrained base as a frozen feature extractor. The setup is described in Table 3.

Table 3
MobileNetV2 base model configuration.

Component / Setting	Description / Value
Base architecture	`tf.keras.applications.MobileNetV2`
Input shape	`(224, 224, 3)`, corresponding to RGB images resized to 224×224 pixels
Pretrained weights	`imagenet`, using weights learned from ImageNet pretraining
Top classification layers	`include_top=False`, meaning the original ImageNet classifier is removed
Base model trainable flag	`False`, so the pretrained MobileNetV2 feature extractor is frozen during training

The complete model applied the shared data augmentation layer, MobileNetV2 preprocessing through `mobilenet_v2.preprocess_input`, the frozen MobileNetV2 base, global average pooling, dropout with rate 0.25, and a dense softmax output layer over the 38 classes.

The frozen-base model was compiled with Adam, learning rate 10^{-3}, categorical cross-entropy, and accuracy. It was trained for up to 8 epochs with the same early-stopping and learning-rate callbacks. After frozen-base training, validation accuracy was approximately 95.0%.

5.3. Fine-Tuning

After the classification head had learned to map MobileNetV2 features to plant disease classes, the model was fine-tuned. Fine-tuning means unfreezing part of the pretrained base and continuing training with a much smaller learning rate. In this project, the last 30 layers of the MobileNetV2 base were made trainable, while earlier layers remained frozen.

The fine-tuning stage settings are described in Table 4.

Table 4
MobileNetV2 fine-tuning configuration.

Fine-tuning Setting	Value / Description
Trainable part	Top 30 layers of the MobileNetV2 base
Frozen part	All earlier layers of the MobileNetV2 base remain frozen
Optimizer	Adam
Learning rate	10^{-5}
Loss	Categorical cross-entropy
Maximum fine-tuning epochs	5

The lower learning rate is important. Pretrained weights should be adjusted carefully because large updates can destroy useful generic visual representations. Fine-tuning only the top feature layers is a compromise: the model can adapt to plant disease patterns without retraining the entire pretrained network.

5.4. Observed Results

Table 5 summarizes the main observed validation results.

Table 5
Observed validation performance from the training notebook.

Model stage	Validation loss	Validation accuracy
Custom CNN baseline	0.2287	0.9330
MobileNetV2 frozen base	0.1519	0.9503
MobileNetV2 fine-tuned	0.1062	0.9653

The fine-tuned MobileNetV2 model was selected as the default runtime classifier because it achieved the best validation performance. The custom CNN remained useful as a second model for comparison and uncertainty handling.

6. SHAP-Based Visual Explanations

A plant disease classifier can report a disease class and confidence score, but these values do not show why the model made its decision. Explainability is therefore important in this project because the system acts not only as a prediction tool, but also as an AI assistant whose behavior should be understandable. A visual explanation helps the user assess whether the model focused on meaningful leaf symptoms, such as spots, discoloration, lesions, or texture changes, rather than background, lighting, or image framing.

SHAP [2] was used to generate these visual explanations. It treats the classifier as a prediction function and estimates how different image regions affect the selected output class. To do this, SHAP repeatedly modifies the image by masking regions and observing how the model prediction changes. Regions that strongly affect the prediction are then highlighted in the explanation.

For image inputs, the masking strategy is important. Simply covering parts of the leaf with a black or solid-colored patch could create unrealistic images and distort the model's behavior. Instead, the SHAP setup uses inpainting, where masked regions are filled using surrounding visual information. This produces more natural perturbations and is better suited to plant images, where texture, color, and local disease patterns matter.

The explanation is generated from the same preprocessed image used for prediction. The model is evaluated many times on perturbed versions of this image, usually focusing on the predicted class. The final output is a heatmap-like visualization that shows which regions most influenced the prediction, helping indicate whether the decision was based on visible disease symptoms or less relevant image features.

7. Plant Disease Knowledge Base and Vector Database

7.1. Why a Vector Database Was Needed

The classifier answers a narrow visual question: which label best matches the uploaded image? However, the user usually needs more than a label. They may ask what symptoms to expect, whether the disease is serious, how to prevent spread, whether to prune infected leaves, or what environmental conditions favor the disease. This information is better represented in text documents than in the CNN itself.

For this reason, the project includes a retrieval-augmented generation component. The retrieval system searches local plant disease resources and returns relevant passages to the agent. The language model can then use these retrieved passages to produce a grounded answer instead of relying only on its parametric memory.

7.2. Document Selection

The knowledge base was built from curated online resources including scientific papers, books and web articles. The resources were mainly crawled from university and academic websites. The sources include general plant pathology references, crop-specific growing guides, and disease-specific extension resources. The selected resources cover both the crops in the image dataset and the diseases represented in the class labels.

7.3. Document Download and Normalization

The selected documents are downloaded and normalized before the chunking phase. Specifically the normalization process will remove from the documents all non useful content and organise all the useful content, such as text of an article or paragraphs of a paper, into a markdown file. Each normalized document is equipped with metadata describing its content and provenance.

7.4. Chunking

The vector database notebook loads all 60 successful normalized Markdown documents. Together they contain 984,286 extracted characters. These documents are then split into chunks with a recursive character splitter. The chunking settings are described in Table 6

Table 6
Settings of the chunking phase.

Parameter	Value
Chunk size	1000 characters
Chunk overlap	200 characters
Minimum chunk length	100 characters
Separators	Section headings, paragraph breaks, line breaks, sentence breaks, spaces, and individual characters

The separator order matters. The splitter first tries to preserve higher-level structure such as Markdown headings and paragraphs. If a section is too long, it falls back to smaller separators. This helps keep chunks semantically coherent while still enforcing the maximum chunk size.

Very short chunks under 100 characters are skipped because they often contain too little information to be useful in retrieval.

7.5. Embeddings and Chroma

The embedding model used to create embeddings for the vector database is `sentence-transformers/all-MiniLM-L6-v2` and it was selected because it is a compact sentence-transformer suitable for local semantic search. It produces dense embeddings that allow Chroma [4] to retrieve chunks based on semantic similarity rather than exact keyword matching. As a vector database Chroma was selected, as it is a very common and easy solution. Specifically it is suitable for this project as it can be stored locally.

The retrieval output is not treated as final user-facing text by itself. Instead, it is passed to the agent. The agent is instructed to ground care and management advice in retrieved results and to mention source titles or URLs when available.

[4] https://www.trychroma.com/products/chromadb

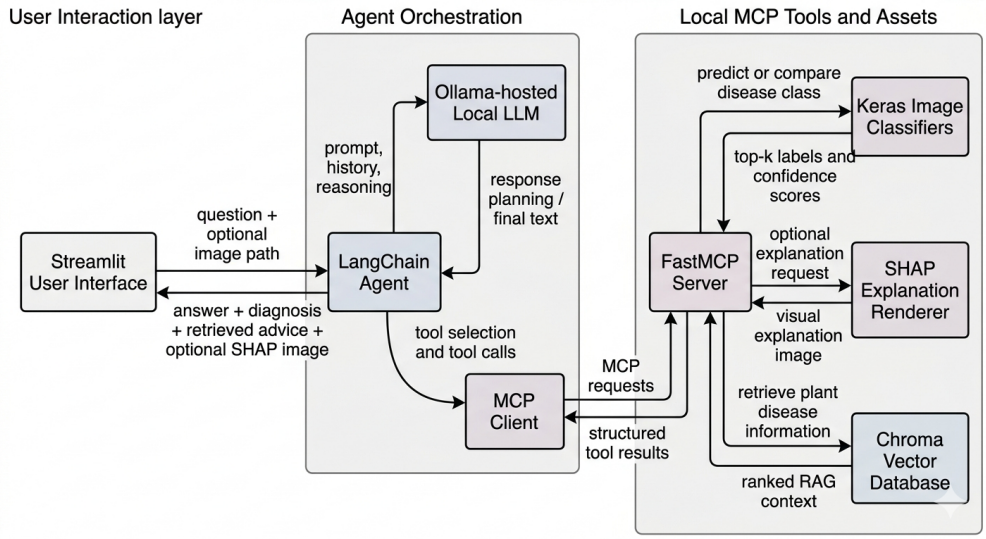

Figure 2: Agentic architecture implemented by the plant disease assistant.

8. Agentic System Design

8.1. Overall Architecture

The final application combines the prepared assets into an agentic system. The main runtime components are highlighted and presented in Fig. 2

The system begins when the user enters a message in Streamlit, optionally attaching a leaf image. If an image is uploaded, Streamlit saves it locally before passing the user question, image path, and compact chat history to the LangChain agent. The agent sends the prompt to the Ollama model with access to MCP tools, which it can call for prediction, comparison, SHAP explanations, or retrieval. The MCP server then loads the required local models or vector database and returns structured results. Finally, the agent produces the response, optionally including retrieved advice or generated explanation images, and Streamlit renders the assistant text together with any returned base64 image content.

This architecture keeps the language model separate from domain-specific computation. The LLM decides which tool to use and writes the final natural-language response, but the actual image classification, model comparison, SHAP computation, and vector search are performed by deterministic Python tools. This perfectly aligns with the paradigm shift, from chatbot to autonomous agents. This agent, as shown, is independent and it is able to select and use the appropriate tools for each situation and request.

9. MCP Tool Interface

MCP provides a clean boundary between the agent and the local capabilities used by the application. Instead of embedding all domain logic inside the prompt, the project exposes plant disease operations as tools with typed arguments and explicit output schemas. This allows the agent to request image classification without directly inspecting image pixels, while keeping validation, model execution, retrieval, and output formatting inside deterministic Python code. The same design also makes the system easier to test, because the tools can be exercised independently from the chat interface, and it keeps the architecture modular enough for the UI, agent, and tool server to evolve separately.

The runtime MCP server exposes three main tools: `predict_plant_disease`, `compare_plant_disease_models`, and `retrieve_plant_disease_info`. Together, these tools

cover image diagnosis, uncertainty checking across models, and retrieval of supporting plant disease information.

9.1. `predict_plant_disease` tool

The `predict_plant_disease` tool classifies a single local plant leaf image. It receives the image path, selects a model, runs preprocessing and inference, and optionally generates a SHAP explanation image. Its main interface is summarized in Table 7.

Table 7
Main arguments for `predict_plant_disease`.

Argument	Purpose
`image_path`	Local path to the image file.
`model_name`	Model to use, defaulting to `MobileNetV2`.
`top_k`	Number of ranked predictions to return.
`include_shap`	Whether to return a SHAP explanation image.
`shap_max_evals`	SHAP perturbation budget.
`shap_batch_size`	Batch size used during SHAP computation.

Before inference, the tool normalizes the model name. Supported names include `MobileNetV2`, `Custom CNN`, and aliases such as `mobilenetv2`, `custom`, and `custom_cnn`. It then loads the class names and selected Keras model, resolves the image path, resizes the image to 224×224, converts it to a floating-point array, and runs prediction.

The structured response records the selected model, resolved image path, top class index, top class name, top confidence, ranked top-k predictions, and SHAP metadata. When `include_shap=True`, the tool also returns MCP image content: the SHAP PNG is base64 encoded and added to the tool result, while the JSON response records the explained class, MIME type, maximum evaluations, and batch size.

9.2. `compare_plant_disease_models` tool

The `compare_plant_disease_models` tool runs both the MobileNetV2 model and the custom CNN on the same image. It is used when the user asks about model agreement, confidence, or uncertainty, and the agent is instructed to call it automatically when the MobileNetV2 top confidence is below 85%.

Its output is designed to make disagreement explicit. The response includes the resolved image path, the requested top-k value, a Boolean agreement flag, one result object for each model, the absolute confidence gap between the two top predictions, and a summary string set to either `models_agree` or `models_disagree`. When both models agree with high confidence, the assistant can present the diagnosis more directly. When they disagree, the assistant should avoid overstating certainty and explain that the local models do not agree on the top class.

9.3. `retrieve_plant_disease_info` tool

The `retrieve_plant_disease_info` tool performs semantic search over the local Chroma vector database. It accepts a natural-language `query` about plants, diseases, symptoms, prevention, care, or management, together with a `top_k` value constrained between 1 and 10.

The retrieval tool loads the persisted Chroma collection `plant_disease_resources` and uses `sentence-transformers/all-MiniLM-L6-v2` as the embedding function. Its structured result contains the original query, the number of returned results, and a ranked set of matching chunks. Each retrieved chunk includes the rank, similarity score, content, title, labels, source URL, final URL, resource ID, and chunk index.

The agent uses this tool after prediction when generating symptoms, prevention, care, or management advice. It can also use the same retrieval path for text-only plant disease questions, allowing the assistant to ground its answer in the local knowledge base even when no image is provided.

9.4. Tool Output Schemas

The tool output schemas are explicit and restrictive. They use `additionalProperties=False` and define required fields, which gives the agent stable structured data and makes the tool contracts easy to inspect in tests and reviews. The prediction, comparison, and retrieval schemas are defined in `src/plant_assistant/mcp/constants.py`.

9.5. Agent connection

A HTTP connection lets the LangChain agent call MCP tools through a local server endpoint, instead of importing the tool functions directly. The agent sends structured tool-call requests over HTTP, the MCP server executes the local Python tool, and the result is returned as structured data that the agent can use in its next reasoning step or final answer.

10. Plant Disease Assistant

10.1. Local LLM

The application uses Ollama to serve the local language model. Ollama runs the model locally and exposes it through an HTTP API, so the rest of the application can treat the model as a chat backend without sending user data to an external hosted provider. The default configured model is `qwen3.6:27B`.

At runtime, the application connects to Ollama through `ChatOllama` from `langchain_ollama`. This wrapper adapts the local Ollama model to LangChain's chat model interface, which means the same agent loop can send messages, receive assistant responses, and process tool calls using LangChain abstractions. Ollama and ChatOllama settings are described int Table 8

Table 8
Main Ollama and `ChatOllama` configuration values.

Setting	Role in the application
`OLLAMA_BASE_URL=localhost:11434`	Points LangChain to the local Ollama server.
`OLLAMA_MODEL=qwen3.6:latest`	Selects the default local chat model.
`temperature=0.2`	Keeps responses stable and less creative.
`reasoning=False`	Disables explicit reasoning output from the model.
`OLLAMA_NUM_CTX=32768`	Provides a large context window for conversation history and retrieved content.
`OLLAMA_NUM_PREDICT=1024`	Limits the maximum generated response length.
`OLLAMA_KEEP_ALIVE=2m`	Keeps the model loaded briefly between requests.
`OLLAMA_REQUEST_TIMEOUT_SECONDS=120`	Prevents a stalled model request from blocking indefinitely.

The low temperature is appropriate for a diagnostic assistant because the answers should be consistent, grounded, and cautious rather than highly creative. The large context window allows the model to see several turns of dialogue, tool results, and retrieved disease information, while the memory system still compresses older history to avoid unnecessary prompt growth.

10.2. LangChain Agent

The agent is created with LangChain's `create_agent`. In this project, the agent combines three parts: the `ChatOllama` model, the MCP tools, and a system prompt that defines how the assistant should behave. The model provides language understanding and response generation, the tools provide access to local plant disease capabilities, and the prompt defines the policy for when those tools should be used.

A LangChain agent works as a controlled loop around the language model. When the user sends a message, LangChain builds the model input from the system prompt, conversation context, and the latest user request. The model then decides whether it can answer directly or whether it needs to call a tool. If a tool is needed, the model emits a structured tool call rather than ordinary final text. LangChain executes that tool, adds the tool result back into the conversation state, and calls the model again. This process can repeat across several steps until the model has enough information to produce the final answer shown to the user.

In this application, that loop is what turns the assistant from a simple chatbot into a tool-using diagnostic workflow. For example, when the user asks about a leaf image, the agent should call `predict_plant_disease` with the current image path. If the prediction confidence is low, the agent should call `compare_plant_disease_models`. If the answer includes care or management advice, the agent should call `retrieve_plant_disease_info` so the final response is grounded in local reference material rather than only in the model's parametric knowledge.

The agent invocation is asynchronous and wrapped in a timeout so that long model generations or slow tool calls do not block the application indefinitely. Agent recursion and timeout are controlled and limited. The recursion limit is important because each tool call adds another step to the agent loop. A normal interaction may need only one prediction call and one retrieval call, but uncertainty handling or SHAP requests can require additional iterations. The limit of 12 leaves room for these workflows while still preventing accidental infinite tool-use loops.

The system prompt is central to the behavior of the agent. It defines the assistant as a concise plant owner assistant that uses conversation history for follow-up questions, but it also reminds the model that image pixels are not stored in chat memory. This matters because later turns may refer to "the current image," and the agent must use the current local image path rather than assuming it can inspect an earlier image directly.

The same prompt also defines the tool policy. It tells the agent to use `predict_plant_disease` for image classification and diagnosis from local image paths, with MobileNetV2 as the default model. It instructs the agent to use `compare_plant_disease_models` when the user asks about model agreement, confidence, or uncertainty, and to compare models automatically when MobileNetV2 confidence is below 85%. When the models disagree, the prompt tells the assistant not to present the diagnosis as certain. For care and management advice, the prompt requires the agent to retrieve supporting plant disease information after prediction and to mention source titles or URLs when available. If the user asks for a visual explanation, the prompt directs the agent to call prediction with `include_shap=True`.

This prompt-based policy is what coordinates the agent's behavior across turns. The model is not only generating text; it is selecting actions, reading structured tool outputs, deciding whether more evidence is needed, and then synthesizing a user-facing answer from the available information.

11. Conversation Memory Management

11.1. Why Memory Needs Compression

The assistant is designed to support multi-turn conversations, where the user may ask follow-up questions about an earlier diagnosis, a retrieved source, or a previously uploaded image. Keeping this continuity is important, but storing the full conversation exactly as it happened would quickly

become inefficient. Previous user messages, assistant responses, tool outputs, retrieval results, and image-related metadata can all consume a large part of the available context window. This is especially true for tool responses, since prediction and retrieval tools often return structured data, confidence scores, JSON fields, and document excerpts.

For this reason, the system uses explicit memory compression. The purpose is not to remember everything verbatim, but to preserve the information that is useful for future reasoning. The assistant should retain facts such as the diagnosed plant disease, the confidence of the prediction, whether models agreed, and which local resources were retrieved. At the same time, bulky intermediate outputs, full tool payloads, and long document chunks are removed from the conversational context once they have been summarized.

11.2. History Conversion

Conversation messages are first stored by the Streamlit interface as part of the session state. Before each new agent call, this stored history is transformed into a compact form suitable for the language model. The conversion keeps only the conversational roles that are relevant for follow-up reasoning, namely user and assistant messages. Internal messages that should not influence later turns are excluded.

The memory budget is controlled through three limits, summarized in Table 9. Together, these limits define how much recent conversation is retained, how large the full compressed memory may become, and how much text from any individual message can be passed forward.

Table 9
Main controls used to keep conversation memory compact.

Memory control	Purpose
Recent turn limit	Keeps the memory focused on the latest part of the conversation.
Total memory budget	Prevents the combined conversation history from becoming too large.
Per-message limit	Ensures that a single long message does not dominate the context.

When a message already has a concise memory summary, that summary is preferred over the original text. Otherwise, the message content is shortened to fit the per-message budget. If the full converted history is still too large, the oldest turns are removed first. This keeps the most recent context available while avoiding unnecessary growth in the prompt.

11.3. Tool Memory Summaries

Tool outputs are also converted into short descriptive summaries before they are reused in later turns. This is important because tool responses are often much more detailed than what the assistant needs for a follow-up answer. For example, a plant disease classifier may return several ranked predictions with confidence values, while a retrieval tool may return multiple document chunks. The assistant usually only needs the final diagnosis, the most important alternatives, and the titles or topics of the retrieved sources.

For classification results, the memory summary records the predicted disease, the confidence level, and the most relevant alternative predictions. For model comparison, it records whether the models agreed and summarizes the top result from each model. For retrieval, it stores the most relevant local resource titles rather than the full retrieved text. This allows later questions such as “What should I do next?” to refer back to the diagnosis and supporting documents without reintroducing the complete tool output into the language model context.

11.4. Image Path Memory

Images are handled differently from text. The language model does not receive previous image pixels as part of conversation memory. Instead, uploaded images are represented by their local file paths. This keeps the memory lightweight while still allowing the agent to refer to an image when a tool call is needed. The image analysis tool can load the actual image from disk, while the language model only needs enough information to decide whether an image should be used.

The system also distinguishes between current and earlier images. Earlier image paths are retained with a caution that they should only be reused if the user explicitly refers to them. This prevents an important multi-turn failure mode, where the assistant might accidentally analyze an old image when the user has moved on to a new text-only question. When a new image is uploaded, its path is inserted into the current user message, and the agent is instructed to use that specific image for prediction, comparison, or explanation.

12. Streamlit User Interface

12.1. Interface Purpose

The Streamlit interface provides the user-facing part of the application. It presents the assistant as an interactive chat system where the user can write questions, optionally attach a plant image, and receive diagnostic or explanatory responses. The interface keeps the interaction deliberately direct: the user enters a message, the application forwards the request to the agent, and the response is rendered back into the chat.

Although the interface is simple from the user's perspective, it coordinates several important tasks in the background. It manages uploaded files, preserves the conversation state across Streamlit reruns, sends requests to the asynchronous agent, and displays both text responses and generated visual outputs.

12.2. Upload Handling

The chat interface supports common image formats, including JPEG, PNG, GIF, WebP, and BMP. When the user uploads an image, the application stores it in the local upload directory and records both the saved file path and the original filename. The local path is important because downstream image-analysis tools need a stable reference to the file on disk, while the original filename is useful for displaying the uploaded content back to the user.

Each chat turn is associated with at most one current image. If multiple files are provided through the upload widget, the interface uses the first one for that message. The uploaded image is shown as a preview above the user's text so that the conversation clearly reflects which image is being discussed.

The interface also handles the common case where the user uploads an image without writing a message. In that situation, the application supplies a default analysis request on the user's behalf. This makes the workflow natural for users who simply want to upload a leaf image and receive a diagnosis without having to phrase an explicit prompt.

12.3. Rendering Assistant Outputs

Assistant responses can contain both ordinary text and generated images. Text is displayed progressively to create a more interactive chat experience, while visual outputs are decoded and rendered as images inside the conversation. This is used for generated explanations such as SHAP visualizations, where the assistant response includes an image payload rather than only written text.

By supporting both response types, the interface can present diagnostic reasoning in a more useful form. The written answer explains the result, confidence, and recommended next steps, while generated figures can show which parts of the image influenced the model's prediction.

12.4. Context Status

The interface includes a small context-status indicator that reports the local model configuration and the estimated remaining context capacity. This information is not part of the plant diagnosis itself, but it improves transparency during interactive use. Since local language models have finite context windows, the user can see when a conversation is becoming long enough that compression or a new chat may be useful.

The remaining-context estimate is derived from prompt-token information when it is available and compared against the configured context-window size. Because this value depends on reported token counts from the model workflow, it should be understood as a practical runtime indicator rather than an exact measurement of semantic memory.

12.5. New Chat and Session State

Streamlit reruns the application script whenever the user interacts with the page, so the interface relies on session state to preserve continuity. Session state keeps the conversation history, upload widget state, context estimate, prompt-token information, and configured context-window size across reruns. Without this persistent state, the chat would lose its history whenever the interface refreshed.

The "New chat" action clears the current conversation and resets the upload and context-related state. This gives the user a clean interaction without requiring a server restart, which is useful when switching to a new plant image or starting an unrelated diagnostic session.

13. Limitations and Future Improvements

13.1. Dataset and Generalization

The image classifier is trained on a curated augmented dataset. Such datasets are useful for controlled experiments, but they may not fully represent real-world images. Real user images can include complex backgrounds, multiple leaves, poor lighting, occlusion, mixed symptoms, or diseases outside the 38 known classes. The model will still choose one of the known classes, even when the true condition is out of distribution.

A future improvement would be to add out-of-distribution detection or a confidence calibration layer. The current system partially addresses uncertainty by comparing MobileNetV2 and the custom CNN when confidence is low, but this is not a complete solution.

13.2. Healthy Versus Diseased Labels

Some crops have healthy classes, while others are represented only by disease classes or a subset of possible conditions. This can affect user expectations. A prediction of a disease class should not be interpreted as a complete plant health diagnosis. It is a model prediction over the classes it was trained to recognize.

13.3. SHAP Runtime Cost

SHAP explanations are useful but expensive. The current implementation makes SHAP optional and uses a configurable evaluation budget. For a smoother user experience, future work could cache explanations for repeated images or explore faster explanation methods such as Grad-CAM.

However, SHAP was appropriate for this project because it is model-agnostic and integrates cleanly with the existing Keras prediction function.

13.4. Retrieval Coverage

The vector database contains 60 successful resources and covers many of the dataset labels, but it is still a curated snapshot. It may not include region-specific recommendations, pesticide regulations, seasonal warnings, or the newest disease-management advice. Because agricultural recommendations can be context-dependent, the assistant should present retrieved advice as informational support rather than a substitute for local expert diagnosis.

13.5. Agent Reliability

The agent depends on the local language model to choose tools correctly and synthesize final answers. The system prompt strongly guides tool use, and tool outputs are structured, but LLM behavior can still vary. Future improvements could add deterministic orchestration for common flows. For example, image diagnosis could always follow a fixed pipeline: predict, compare if low confidence, retrieve disease information, then ask the LLM only to summarize.

13.6. User Interface

The Streamlit interface is functional and suitable for demonstration, but future versions could improve usability. Possible improvements include showing a structured diagnosis card, separating prediction confidence from advice, displaying retrieved sources in an expandable section, adding a model-comparison panel, and allowing the user to select between MobileNetV2 and the custom CNN.

14. Conclusion

This project implemented a local plant disease assistant that combines computer vision, explainable AI, retrieval, and agentic interaction. The preparatory phase produced two trained Keras classifiers and a persisted Chroma vector database. The custom CNN provided a baseline with about 93.3% validation accuracy, while the fine-tuned MobileNetV2 model achieved about 96.5% validation accuracy and became the default classifier. SHAP was integrated to provide optional visual explanations for model predictions.

The runtime system exposes prediction, model comparison, SHAP, and retrieval through FastMCP tools. A LangChain agent using a local Ollama model decides when to call these tools and synthesizes final answers. Conversation memory is compressed so follow-up questions remain possible without filling the context window with raw tool JSON. A Streamlit interface ties the system together by accepting chat messages and image uploads, rendering assistant text, and displaying SHAP explanation images.

The main contribution of the project is therefore not only a trained image classifier, but an end-to-end assistant architecture. The system moves from leaf image recognition to grounded plant disease advice through a modular combination of CNN models, explainability, vector search, MCP tools, a local LLM, and an interactive UI.

References

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobilenetv2: Inverted residuals and linear bottlenecks, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 4510–4520.

[2] S. M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Advances in neural information processing systems 30 (2017).